

ARM PrimeCell Driver

platfm v1.7 (System Configuration)

API Specification

Primecell driver API description

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	E
Primecell driver identifier	platfm
Code Version	1.7

Contents

1	ABOUT THIS DOCUMENT	5
1.1	Change control	5
1.1.1	Change history	5
2	SUMMARY	6
3	HARDWARE INTEGRATION	6
3.1	Declaring peripheral instances	6
3.1.1	Adding a new instance of a peripheral	6
3.2	Specifying the base address of peripherals	7
3.2.1	Changing the base address of a peripheral	8
3.2.2	Adding the base address of an additional peripheral	8
3.3	Interrupt source format	9
3.4	Specifying the bit position of interrupt sources	9
3.4.1	Changing the bit position of an interrupt source	10
3.4.2	Adding the bit position of an additional interrupt source	10
3.5	Specifying a PrimeCell variant	11
4	DETAILS	12
5	RELEASED FILES	12
6	PUBLIC API	13
6.1	Type Definitions	13
6.1.1	apOS_GPIO_old	13
6.1.2	apOS_INT_old	13
6.1.3	apOS_INT_oInterruptSource	13
6.1.4	apOS_RTC_old	13
6.1.5	apOS_System_eBaseAddress	14
6.1.6	apOS_TIMER_old	14
6.1.7	apOS_UART_old	15
6.2	Macros	15
6.2.1	TEST_ALL_MODULES	15
6.2.2	apOS_CONFIG_USE_NO_CLIB	15
6.2.3	apOS_INTCTL_AP	15
6.2.4	apOS_INTCTL_LM	16
6.2.5	apOS_PLATFORM_HAS_TIMERS	16
6.2.6	apOS_PLATFORM_INTEGRATOR	16

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A		First issue of API
B	15/11/00	Revised document to match desired template format
C	20/03/01	Removed interrupt configuration item

2 SUMMARY

The system configuration module is a 'C' header file holding the user-configured details of the system hardware configuration.

By grouping this information into one location, specifying a system configuration is simplified.

This file is user-configurable to specify the Primecells present in the system, their base addresses and interrupts used.

3 HARDWARE INTEGRATION

This chapter describes the changes and additions you must make to the supplied software if you make changes or additions to the default hardware supplied.

3.1 Declaring peripheral instances

For each peripheral that has multiple instances, there is an enumerated type that:

- ◆ declares each instance of the peripheral
- ◆ declares the total number of available instances of the peripheral.

Example 1 shows the general format of these enumerated types for a generalised driver **MOD**.

Example 1 - Format of enumerated types to declare peripheral instances

```
typedef enum apOS_MOD_xId
{
    apOS_MOD_0 = 0,      // First instance of peripheral
    apOS_MOD_1 = 1,      // Second instance of peripheral
    ...                  // Further instances of peripheral
    apOS_MOD_MAXIMUM    // Number of available peripherals
}
apOS_MOD_oId;
```

In Example 1, the enumerator apOS_**MOD**_MAXIMUM does not have a value explicitly assigned to it. Its value is therefore that of the previous enumerator, incremented by one.

This syntax ensures that its value is always correct, if the peripheral instances are always numbered consecutively from 0.

3.1.1 Adding a new instance of a peripheral

1. Add a new enumerator declaring the new instance of the peripheral.
2. Ensure that the enumerator apOS_**MOD**_MAXIMUM giving the number of available instances remains correct.

The enumerated type listing the UARTs is apOS_UART_old. Its declaration in the example header file is shown in Example 2.

Example 2 - UART enumeration

```
typedef enum apOS_UART_xId
{
    apOS_UART_0 = 0,           // Specifies UART 0
    apOS_UART_1 = 1,           // Specifies UART 1
    apOS_UART_MAXIMUM           // Number of UARTs in the system
}
apOS_UART_oId;                //Enumeration of UARTs available for the system
```

To declare the extra UART, you add an enumerator apOS_UART_2, as shown in Example 3.

Example 3 – Revised UART enumeration

```
typedef enum apOS_UART_xId
{
    apOS_UART_0 = 0,           // Specifies UART 0
    apOS_UART_1 = 1,           // Specifies UART 1
    apOS_MY_UART = 2,          // Specifies new UART added to system
    apOS_UART_MAXIMUM           // Number of UARTs in the system
}
apOS_UART_oId;                //Enumeration of UARTs available for the system
```

The value of the enumerator apOS_UART_MAXIMUM is now implicitly set to 3. It is still giving the correct number of UARTs in the system, and so no more editing is necessary.

3.2 Specifying the base address of peripherals

The enumerated type apOS_System_eBaseAddress specifies the base address of various peripherals. The value of each enumerator is the address, in bytes.

The enumerators for this type are defined in the file **applatfm.h**, and the shipped values are shown in Example 4

Example 4 - Enumerators for specifying the base address of peripherals

```
typedef enum apOS_System_eBaseAddress
{
    apOS_SYSTEM_BASE_TIMER0      = (WORD32) 0x13000000, // Timer 0
    apOS_SYSTEM_BASE_TIMER1      = (WORD32) 0x13000100, // Timer 1
    apOS_SYSTEM_BASE_TIMER2      = (WORD32) 0x13000200, // Timer 2
    apOS_SYSTEM_BASE_INT         = (WORD32) 0x14000000, // Interrupt controller
    apOS_SYSTEM_BASE_RTC         = (WORD32) 0x15000000, // Real Time Clock
    apOS_SYSTEM_BASE_UART0       = (WORD32) 0x16000000, // UART 0
    apOS_SYSTEM_BASE_UART1       = (WORD32) 0x17000000, // UART 1
    apOS_SYSTEM_BASE_KMI0        = (WORD32) 0x18000000, // Keyboard/Mouse Interface 0
    apOS_SYSTEM_BASE_KMI1        = (WORD32) 0x19000000, // Keyboard/Mouse Interface 1
    apOS_SYSTEM_BASE_GPIO        = (WORD32) 0x1B000000, // General Purpose IO
}
apOS_System_eBaseAddress;
```

3.2.1 Changing the base address of a peripheral

If you change the base address of a peripheral, you must change the value of the corresponding enumerator.

For example, if you change the base address of one UART to 0x72000000, you must change the value of apOS_SYSTEM_BASE_UART0, as shown in Example 5.

Example 5 - Changing the base address of UART peripheral

```
typedef enum apOS_System_eBaseAddress
{
    apOS_SYSTEM_BASE_TIMER0      = (WORD32) 0x13000000, // Timer 0
    apOS_SYSTEM_BASE_TIMER1      = (WORD32) 0x13000100, // Timer 1
    apOS_SYSTEM_BASE_TIMER2      = (WORD32) 0x13000200, // Timer 2
    apOS_SYSTEM_BASE_INT         = (WORD32) 0x14000000, // Interrupt controller
    apOS_SYSTEM_BASE_RTC         = (WORD32) 0x15000000, // Real Time Clock
    apOS_SYSTEM_BASE_UART0       = (WORD32) 0x72000000, // UART 0 at new address
    apOS_SYSTEM_BASE_UART1       = (WORD32) 0x17000000, // UART 1
    apOS_SYSTEM_BASE_KMI0        = (WORD32) 0x18000000, // Keyboard/Mouse Interface 0
    apOS_SYSTEM_BASE_KMI1        = (WORD32) 0x19000000, // Keyboard/Mouse Interface 1
    apOS_SYSTEM_BASE_GPIO        = (WORD32) 0x1B000000, // General Purpose IO
}
apOS_System_eBaseAddress;
```

3.2.2 Adding the base address of an additional peripheral

If you add a new peripheral, you must add a corresponding enumerator to specify its base address. The enumerator must be named apOS_SYSTEM_BASE_PERIPHERAL, where PERIPHERAL is the name of the peripheral, in uppercase.

For example, if you add an extra UART, you must add an enumerator to specify its base address, as shown in Example 6.

Example 6 - Adding the base address of an additional UART

```
typedef enum apOS_System_eBaseAddress
{
    apOS_SYSTEM_BASE_TIMER0      = (WORD32) 0x13000000, // Timer 0
    apOS_SYSTEM_BASE_TIMER1      = (WORD32) 0x13000100, // Timer 1
    apOS_SYSTEM_BASE_TIMER2      = (WORD32) 0x13000200, // Timer 2
    apOS_SYSTEM_BASE_INT         = (WORD32) 0x14000000, // Interrupt controller
    apOS_SYSTEM_BASE_RTC         = (WORD32) 0x15000000, // Real Time Clock
    apOS_SYSTEM_BASE_UART0       = (WORD32) 0x16000000, // UART 0
    apOS_SYSTEM_BASE_UART1       = (WORD32) 0x17000000, // UART 1
    apOS_SYSTEM_BASE_MY_UART     = (WORD32) 0x72000000, // UART 2 added
    apOS_SYSTEM_BASE_KMI0        = (WORD32) 0x18000000, // Keyboard/Mouse Interface 0
    apOS_SYSTEM_BASE_KMI1        = (WORD32) 0x19000000, // Keyboard/Mouse Interface 1
    apOS_SYSTEM_BASE_GPIO        = (WORD32) 0x1B000000, // General Purpose IO
}
apOS_System_eBaseAddress;
```

3.3 Interrupt source format

The interrupt source format is opaque to the driver modules. The source is specified by a single word that is passed to the interrupt controller. The interpretation of this word depends on the design of the interrupt controller.

The format used by the supplied interrupt controller is:

- ◆ Bits 8-15 specify the controller to be used as an enumeration value (see §3.1)
- ◆ Bits 0-7 specify the interrupt source to be passed to that interrupt controller

3.4 Specifying the bit position of interrupt sources

The enumerated type `apOS_INT_oInterruptSource` specifies the bit position of each interrupt source.

The enumerators for this type are defined in the file **applatfm.h**, and a section of the shipped values are shown in Example 7. Here the interrupt controller is defined as a constant in bits 8-15 as below.

Example 7 - Enumerated type for interrupt bit positions

```
#define apOS_INTCTL_AP (0<<8)           //interrupt controller for Integrator AP
#define apOS_INTCTL_LM (1<<8)           //interrupt controller for Integrator LM

typedef enum apOS_INT_xInterruptSource
{
    apOS_INT_PROGRAMMED = apOS_INTCTL_AP | 0x00, // Software interrupt source
    apOS_INT_UART0      = apOS_INTCTL_AP | 0x01, // UART 0 interrupt source
    apOS_INT_UART1      = apOS_INTCTL_AP | 0x02, // UART 1 interrupt source
    apOS_INT_KMI0       = apOS_INTCTL_AP | 0x03, // Keyboard/Mouse interrupt source
    apOS_INT_KMI1       = apOS_INTCTL_AP | 0x04, // Keyboard/Mouse interrupt source
    .
    .
    .
    apOS_INT_NONE       = 0x00000020, // No source - for testing only
    apOS_INT_32BIT      = 0x7FFFFFFF  // Forces compiler to use 32-bit int for enum
}
apOS_INT_oInterruptSource;
```

3.4.1 Changing the bit position of an interrupt source

If you change the bit position of an interrupt source, you must change the value of the corresponding enumerator.

For example, if you change the interrupt source for UART 0 to use bit 17, you must change the value of `apOS_INT_UART0` to also be 17, as shown in Example 8.

Example 8 - Changing the bit position of a timer interrupt source

```
typedef enum apOS_INT_xInterruptSource
{
    apOS_INT_PROGRAMMED = apOS_INTCTL_AP | 0x00, // Software interrupt source
    apOS_INT_UART0      = apOS_INTCTL_AP | 0x11, // Modified UART 0 interrupt source
    apOS_INT_UART1      = apOS_INTCTL_AP | 0x02, // UART 1 interrupt source
    apOS_INT_KMI0       = apOS_INTCTL_AP | 0x03, // Keyboard/Mouse interrupt source
    apOS_INT_KMI1       = apOS_INTCTL_AP | 0x04, // Keyboard/Mouse interrupt source
    .
    .
    apOS_INT_NONE       = 0x00000020, // No source - for testing only
    apOS_INT_32BIT      = 0x7FFFFFFF // Forces compiler to use 32-bit int for enum
}
apOS_INT_oInterruptSource;
```

3.4.2 Adding the bit position of an additional interrupt source

If you add an additional interrupt source, for example by adding a new peripheral, you must add a corresponding enumerator. The enumerator must be named `apOS_INT_PERIPHERAL`, where `PERIPHERAL` is the name of the peripheral, in uppercase.

For example, if you add an extra UART, you must add an enumerator named `apOS_INT_UART2` to specify the bit position of its interrupt source, as shown in Example 9.

Example 9 - Adding the bit position of an additional UART interrupt source

```
typedef enum apOS_INT_xInterruptSource
{
    apOS_INT_PROGRAMMED = apOS_INTCTL_AP | 0x00, // Software interrupt source
    apOS_INT_UART0      = apOS_INTCTL_AP | 0x01, // UART 0 interrupt source
    apOS_INT_UART1      = apOS_INTCTL_AP | 0x02, // UART 1 interrupt source
    apOS_INT_MY_UART    = apOS_INTCTL_AP | 0x11, // Added UART 2 interrupt source
    apOS_INT_KMI0       = apOS_INTCTL_AP | 0x03, // Keyboard/Mouse interrupt source
    apOS_INT_KMI1       = apOS_INTCTL_AP | 0x04, // Keyboard/Mouse interrupt source
    .
    .
    apOS_INT_NONE       = 0x00000020, // No source - for testing only
    apOS_INT_32BIT      = 0x7FFFFFFF // Forces compiler to use 32-bit int for enum
}
apOS_INT_oInterruptSource;
```

3.5 Specifying a PrimeCell variant

Some PrimeCell drivers will support a range of PrimeCell variants, selected at compile time. Refer to the driver API document to determine if this is the case.

For example, the UART driver will support the PL010, PL011 and the Integrator AP UART.

If installing such a driver, the variant used should be specified in the file **applatfm.h**.

A variant PLxxx for driver **MOD** is chosen by `#define` of a pre-processor constant `apMOD_VERSION`, setting this to value **xxx** as in Example 10.

Example 10 - Selecting UART type PL011

```
#define apUART_VERSION 11
```

4 DETAILS

- ◆ Peak performance obtained in tests: N/A
- ◆ ROM size (RO code + RO data) (release build): 0 Bytes
- ◆ RAM size (ZI data) per instance (release build): 0 Bytes
- ◆ Does this code support both endian-nesses?¹ YES
- ◆ Does this code support Thumb?² YES

5 RELEASED FILES

PL000-ISPC-1001-E

apos/applatfm.h

This API document

Self-test module

¹ To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

² The driver may include ARM code, but this must interwork with a Thumb build.

6 PUBLIC API

6.1 Type Definitions

6.1.1 apOS_GPIO_old

Declaration

```
typedef enum apOS_GPIO_oId
```

Elements

apOS_GPIO_0	General GPIO available to OEM
apOS_GPIO_1	General GPIO available to OEM
apOS_GPIO_MAXIMUM	Number of GPIOs in the system

Remarks

Enumeration of GPIOs available for the system

6.1.2 apOS_INT_old

Declaration

```
typedef enum apOS_INT_oId
```

Elements

apOS_INT_0	The default interrupt controller
apOS_INT_LM	The logic module INTC
apOS_INT_MAXIMUM	Number of interrupt controllers in the system

Remarks

Enumeration of interrupt controllers available for the system.

6.1.3 apOS_INT_oInterruptSource

Enumeration of the bit positions of the various interrupt sources in the each of the interrupt controller registers. Interrupt sources are declared for the standard Integrator AP peripherals.

Declaration

```
typedef enum apOS_INT_oInterruptSource
```

Remarks

This is an opaque type so that the interrupt source can be identified by whatever means the system requires. The implementation used here specifies the controller in bits 8-11, and the source in bits 0-7

6.1.4 apOS_RTC_old

Declaration

```
typedef enum apOS_RTC_oId
```

Elements

apOS_RTC_0	Specifies Timer 0
apOS_RTC_MAXIMUM	Number of timers in the system

Remarks

Enumeration of real time clocks available for the system

6.1.5 apOS_System_eBaseAddress

Enumeration of the base addresses for the various peripheral modules. Base addresses here are for the standard integrator AP peripherals.

Declaration

```
typedef enum apOS_System_eBaseAddress
```

Elements

apOS_SYSTEM_BASE_GPIO	General Purpose IO
apOS_SYSTEM_BASE_INT	Interrupt controller base address
apOS_SYSTEM_BASE_INTLM	LM Interrupt controller base address
apOS_SYSTEM_BASE_KEYBOARD	Keyboard
apOS_SYSTEM_BASE_LM	LM base address (LED's etc)
apOS_SYSTEM_BASE_MOUSE	Mouse
apOS_SYSTEM_BASE_RTC	Real Time Clock
apOS_SYSTEM_BASE_TIMER0	Timer 0
apOS_SYSTEM_BASE_TIMER1	Timer 1
apOS_SYSTEM_BASE_TIMER2	Timer 2
apOS_SYSTEM_BASE_UART0	UART 0
apOS_SYSTEM_BASE_UART1	UART 1

6.1.6 apOS_TIMER_old

Declaration

```
typedef enum apOS_TIMER_oId
```

Elements

apOS_TIMER_0	Specifies Timer 0
apOS_TIMER_1	Specifies Timer 1
apOS_TIMER_2	Specifies the watchdog timer
apOS_TIMER_MAXIMUM	Number of timers in the system

Remarks

Enumeration of timers available for the system

6.1.7 apOS_UART_old

Declaration

```
typedef enum apOS_UART_oId
```

Elements

apOS_UART_0	Specifies UART 0
apOS_UART_1	Specifies UART 1
apOS_UART_MAXIMUM	Number of UARTs in the system

Remarks

Enumeration of UARTs available for the system

6.1.8 apOS_VIC_old

Declaration

```
typedef enum apOS_VIC_oId
```

Elements

apOS_VIC_0	The vectored interrupt controller
apOS_VIC_MAXIMUM	Number of vectored interrupt controllers in the system

Remarks

Enumeration of vectored interrupt controllers available for the system (if any).

6.2 Macros

6.2.1 TEST_ALL_MODULES

Test sequence to be performed for this system.

See the Integration Guide PL000_DII_1000 for details of how to specify tests.

6.2.2 apOS_CONFIG_USE_NO_CLIB

Define the constant apOS_CONFIG_USE_NO_CLIB{xe "apOS_CONFIG_USE_NO_CLIB"} as non-zero if the code should not use any C library functions.

Declaration

```
#define apOS_CONFIG_USE_NO_CLIB FALSE
```

6.2.3 apOS_INTCTL_AP

interrupt controller for Integrator AP

Declaration

```
#define apOS_INTCTL_AP (0<<8)
```

Remarks

Identifiers used in this header to build the interrupt source IDs. Each should be in the second byte of the word

6.2.4 apOS_INTCTL_LM

interrupt controller for Integrator LM

Declaration

```
#define apOS_INTCTL_LM (1<<8)
```

Remarks

Identifiers used in this header to build the interrupt source IDs. Each should be in the second byte of the word

6.2.5 apOS_PLATFORM_HAS_TIMERS

Define the constant apOS_PLATFORM_HAS_TIMERS as false if the platform does not include any timers. This will ensure that no cross-module calls using timers are built.

Declaration

```
#define apOS_PLATFORM_HAS_TIMERS TRUE
```

6.2.6 apOS_PLATFORM_INTEGRATOR

Define the constant apOS_PLATFORM_INTEGRATOR{xe "apOS_PLATFORM_INTEGRATOR"} as non-zero if running on Integrator. This will allow the test code to check for a logic module and chain the interrupt handler

Declaration

```
#define apOS_PLATFORM_NOT_INTEGRATOR FALSE
```